AquaZeroFit

AI-powered wellness — on the open web and inside Telegram, from one codebase.

- **Victor Hong** Nutrition & Dietary System
- **Eric La** Training & Workout System
- **Babatundji "Tundji" Williams-Fulwood** Platform, AI, Safety & Operations — Lead Software Architect

NIT3004 IT Capstone Project 2 · Group 15 · Victoria University, Melbourne

RUNNING ORDER | PRESENTER: WHOLE TEAM

Ten minutes, three engineers, one system

VH

VICTOR HONG

Nutrition & Dietary System

- Calorie & macro engine with a safety floor
- Photo, barcode and one-tap logging
- Allergen filtering the AI cannot override

~2:30

EL

ERIC LA

Training & Workout System

- Licensed exercise corpus
- Deterministic plan generator & progression
- Adaptive readiness and the session logger

~2:30

TW

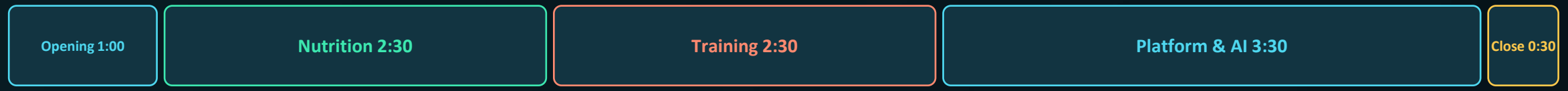
TUNDJI WILLIAMS-FULWOOD

Platform, AI, Safety & Operations

- The AI Gateway and offline resilience
- Safety, security and privacy architecture
- Quality: 770 automated tests

1:00 + ~3:30

THE 10-MINUTE CLOCK



Followed by 5 minutes of Q&A — every slide names its presenter, so questions can go straight to the person who built that part.

One app that knows your numbers — and never guesses them

AquaZeroFit calculates a personal daily calorie, macro and hydration target, lets you log meals by photo, barcode, chat or a single tap, generates a training plan for the equipment you actually own, and answers questions through Aqua Coach — an AI assistant that only ever speaks from your real data.

A DAY WITH THE APP

Morning

Photograph breakfast. A draft comes back in seconds — you correct a portion and confirm before anything is saved.

Midday

Scan a packaged lunch. The barcode is validated and the label's calories are cross-checked, not trusted.

Afternoon

Today's workout is already resolved — and quietly eased, because last week was a hard week.

Evening

Ask the coach how the week is going. It answers with your numbers, because you consented to that.

The one rule every other decision follows

“Models identify. Code calculates.”

An AI can look at a plate and say “that’s rice and grilled chicken”. It cannot be trusted to say “412 calories” — it will produce a fluent, confident, wrong number. Applied to someone’s food every day for months, that is not a software bug. It is a health outcome.

THE MODEL MAY

- Name a food it sees in a photo or reads in a sentence
- Estimate a portion — which code then clamps to a sane range
- Rank options that code has already declared safe
- Phrase a summary of statistics code has already computed

THE MODEL MAY NEVER

- Write a calorie or macro figure into your diary
- Decide whether a food is safe for your allergy
- Decide how much training you are prescribed
- Move a credit, grant an entitlement, or unlock a coach

Why we built a Telegram Mini App

Zero-install distribution

The app opens inside a chat like a message — no app store, no download, no sign-up form. Telegram identity signs you in, verified cryptographically on our server.

Payments built in

Coach unlocks are Telegram Stars invoices. The server sets every price and verifies every payment webhook — the client is never trusted.

Feels native, stays safe

The app adopts the host's theme colours — but only after a WCAG contrast check passes. It uses haptics and deep links like a first-class Telegram citizen.

Still a real website

The same codebase serves a full responsive web app. The Telegram SDK loads only on a genuine Mini App launch, so the public site stays fast for everyone else.

1	2	0	Stars
codebase	delivery surfaces	installs required	in-chat payments

The stack — and why each piece earned its place

TypeScript, end to end

One language for client, server and shared packages. A change to an API shape breaks the other side at compile time — not in front of a user.

React 18 + Vite

Fast, component-based UI with a build step we extended ourselves — our own plugin prerenders real HTML for search engines.

Node.js + Express

A small, well-understood API framework whose middleware chain maps one-to-one onto our seven-step AI safety sequence.

zod validation

One schema in a shared package validates every request on the server and types the client. The contract cannot silently drift.

Tailwind + design tokens

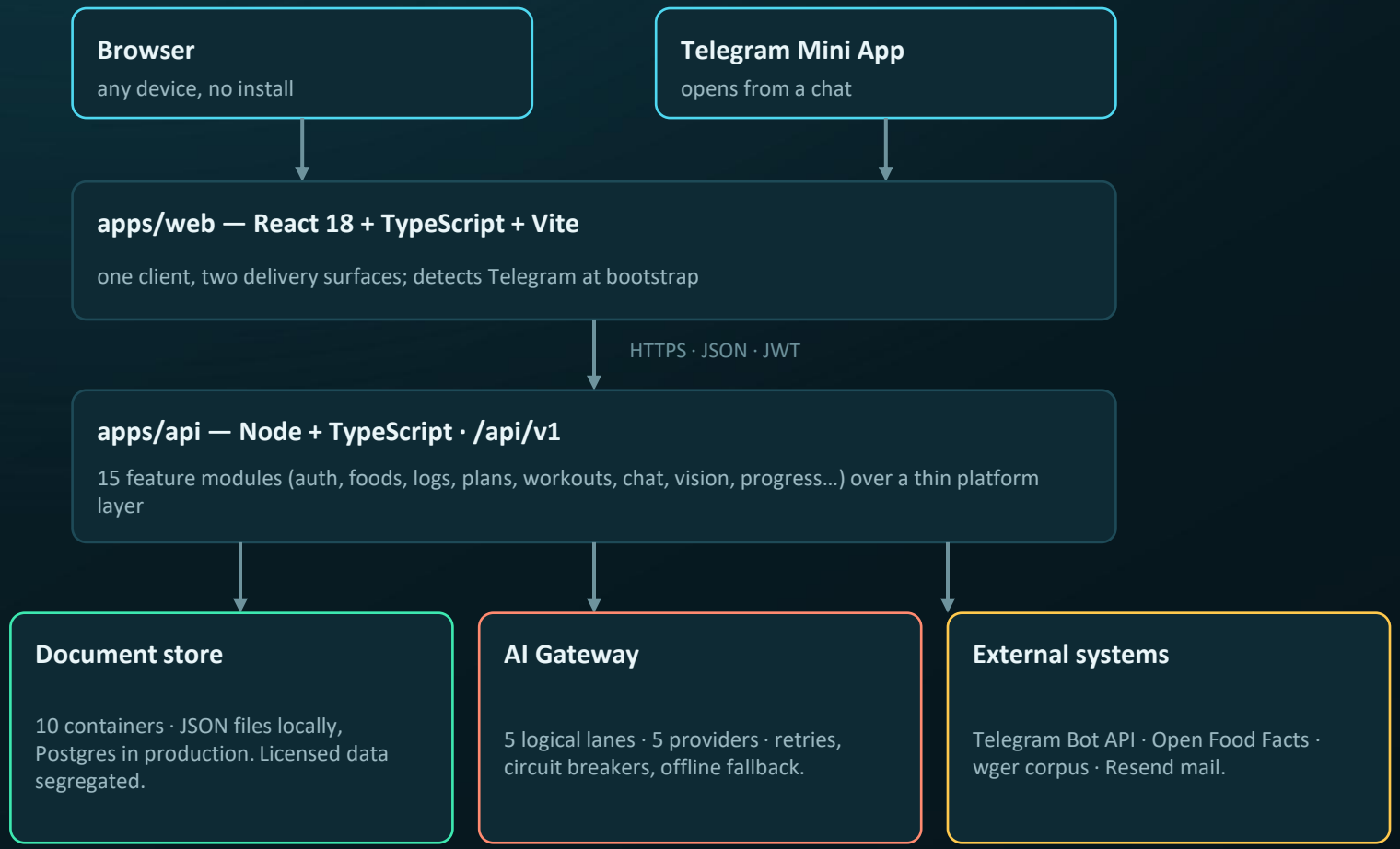
Every colour lives in one token layer — which is exactly what makes Telegram theme binding possible without touching components.

Vitest — 770 tests

One test runner across all three workspaces, fast enough to run the whole suite on every change.

Everything is TypeScript on purpose: three engineers reviewing each other’s work in one shared vocabulary, with one type system keeping all of it honest.

The whole system on one slide



WHY IT IS SHAPED THIS WAY

Modules stay in their lane

No module ever touches another's storage — they talk through service functions.

One door to the AI

Every model call passes the same 7-step admission sequence. No exceptions, no fast paths.

Licences never mix

Open-licence food data lives in its own containers so it can never contaminate our curated corpus.

Your daily targets are arithmetic — never an AI guess

Science first, in deterministic code

Basal rate via the Mifflin-St Jeor equation, scaled by an activity factor (1.2–1.9), then adjusted toward your goal only within a safe band — 0.5–1% of bodyweight per week.

A floor the app will not cross

Targets are clamped to 1,200 kcal (1,500 for men). When the clamp fires, the app tells you why — it never silently overrules you, and never lets you under-eat by design.

Macros and hydration included

Protein set per kilogram by goal (1.6–2.2 g/kg), fat at least 20% of energy, carbohydrate takes the rest, water at 33 ml per kg.

Every number is accountable

The formula version is stored with every set of targets, so we can always say exactly which mathematics produced a figure a user acted on.

1,200 kcal	1.2–1.9	2.0 g/kg	33 ml/kg
safety floor (1,500 men)	activity factors	protein (weight-loss goal)	hydration target

Logging a meal takes seconds — four ways in

Photo → draft → confirm

The model only names the food; the app recomputes calories from its own 139-record corpus. Nothing is saved until you confirm — then the photo is deleted, its GPS and camera metadata already stripped on upload.

Barcodes are checked, not trusted

EAN-8/13 check-digit validated locally, our mirror first, Open Food Facts as fallback — then the label's calories are recomputed from its own macros, and a label that lies outside 30% tolerance is flagged.

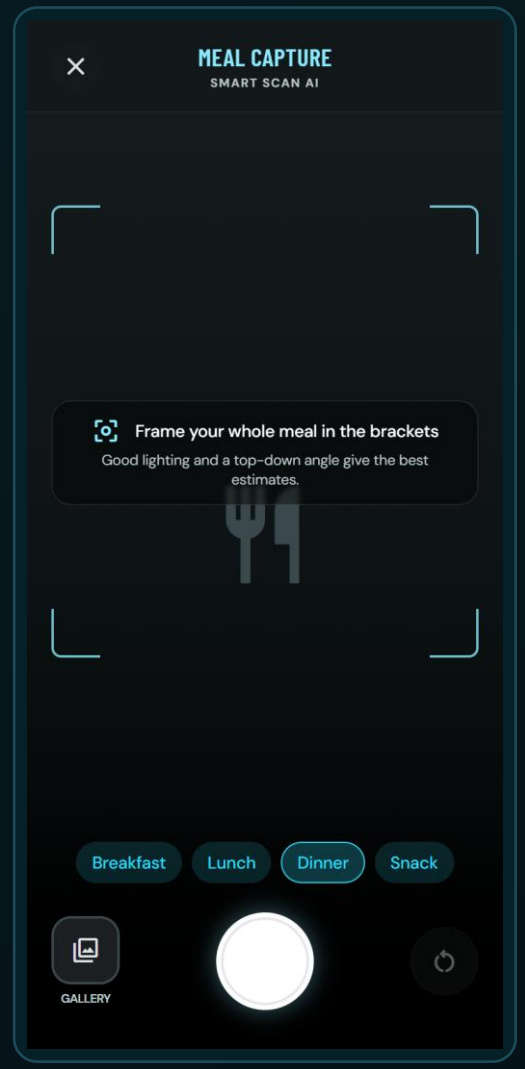
History does the typing

"Copy yesterday" duplicates a whole day of meals in one request, and a month calendar steps back through everything you've logged.

Retry-safe by design

Every creation route honours an idempotency key — a flaky connection can retry forever and never double-log a meal.

139	10 MB	30% + 5 kcal	24 h
foods, each with source + licence	JPEG / PNG / HEIC uploads	label cross-check tolerance	orphaned photos swept



The safety net you never see

Allergens: two independent nets, zero AI

Nine allergen classes filtered in code — the record's declared field, plus a keyword net that catches untagged records (satay → peanuts, tahini → sesame). Over-blocking is acceptable; under-blocking never is.

Suggestions the model cannot sabotage

Code computes your remaining budget and filters candidates for diet and allergens; the model only ranks the survivors — and even its pick is re-checked against the safe list.

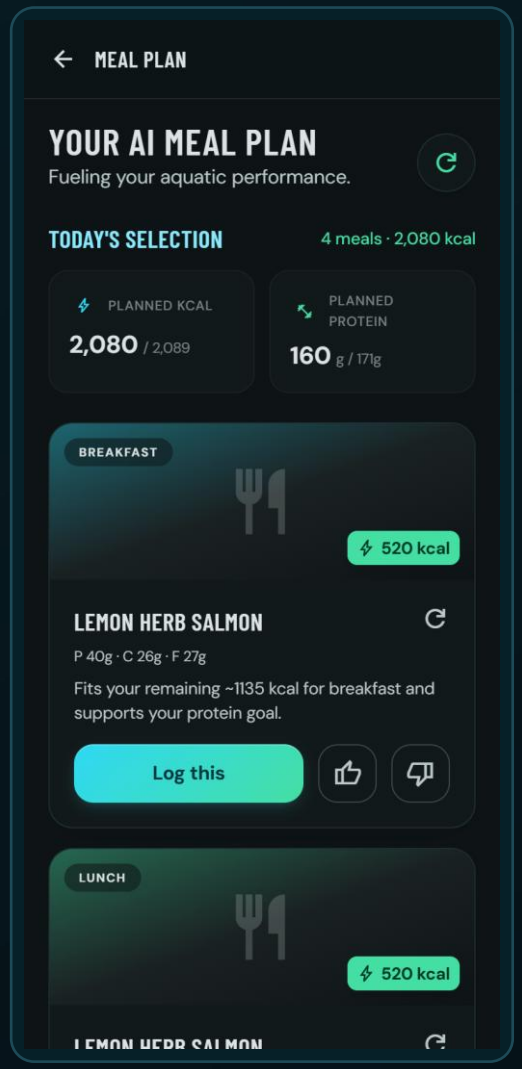
Depth without clutter

Six micronutrients — fibre, sugar, sodium, potassium, calcium, iron — folded live from your log; 17 curated, licensed recipes loggable in one tap.

Your data leaves with you

Full diary export in JSON or CSV — meals, macros, micronutrients, hydration — ready for a dietitian or another app. Data rights you can exercise, not just read about.

9	2	6	JSON/CSV
allergen classes	independent filter nets	micronutrients tracked	one-request export



Where the safety lives in code

apps/api/src/modules/me/targets.ts

```
export function computeTargets(profile) {
  const bmr    = computeBmr(profile);            // Mifflin-St Jeor
  const tdee   = bmr * ACTIVITY_FACTORS[profile.activityLevel];
  const rawTarget = tdee + dailyAdjustment(profile.goal, ...);

  // Safety clamp: the engine never proposes intake below
  // the floor (FR-031).
  const floor      = KCAL_FLOOR[profile.sex];    // 1200 / 1500
  const clamped    = rawTarget < floor;
  const kcalTarget = clamped ? floor : rawTarget;
  const clampReason = clamped
    ? `Calculated target (... kcal) is below the safe
       minimum of ${floor} kcal, so the target was raised
       to the floor. Consider a gentler rate of change.`
    : null;

  return { ..., clamped, clampReason,
           formulaVersion: FORMULA_VERSION };  // mifflin-stjeor-v1
}
```

WHY IT MATTERS No AI import anywhere in this file. The target is pure arithmetic — and when it overrules the user, it says why, in a sentence stored beside the number.

apps/api/src/modules/recommendations/allergenFilter.ts

```
/**
 * Deterministic allergen exclusion (AQF-11 §2: zero
 * tolerance for false negatives). This filter runs in
 * CODE, after candidate selection and before any model
 * involvement — a model NEVER decides allergen safety.
 *
 * Two independent nets:
 *  1. The record's declared `allergens` field.
 *  2. A name/ingredient keyword fallback map that catches
 *     records with missing tagging ("satay" → peanuts).
 * False positives (over-blocking) are acceptable;
 * false negatives are not.
 */
export const ALLERGEN_NAME_KEYWORDS = {
  peanuts:  ['peanut', 'satay', 'groundnut', ...],
  treeNuts: ['almond', 'praline', 'marzipan', ...],
  milk:     ['whey', 'casein', 'ghee', 'halloumi', ...],
  ...       // 9 classes, both nets applied in code
};
```

WHY IT MATTERS The safety policy is written at the top of the file that enforces it — and a test fails if either net stops firing.

A movement library with a licence, not just a list

51 seeded movements, extended from wger

Each categorised strength / cardio / mobility / core, with primary and secondary muscles, required equipment, difficulty and demonstration media.

Attribution survives every refactor

The upstream corpus is Creative Commons share-alike, so licence, author and deed URL are fields on the record itself — never stripped, and rendered to the user in the app.

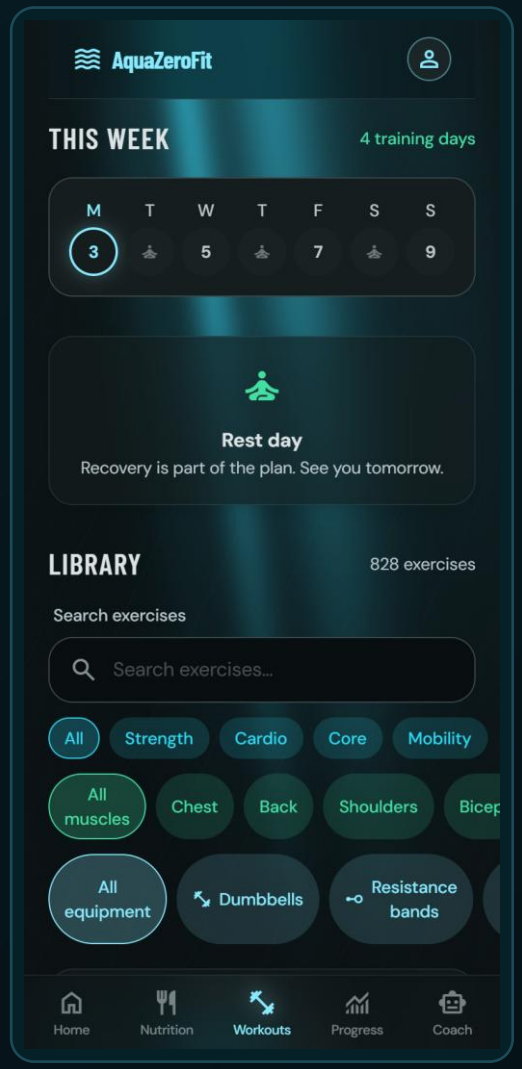
Variation groups

Interchangeable movements share a group ID. That one field is what makes safe in-session swaps possible without breaking historical progression.

Filtered to your reality

Plans only ever draw from movements whose equipment you actually own and whose difficulty matches your experience.

51 + wger	CC BY-SA	4
seeded & imported movements	honoured on every record	movement categories



Plans are deterministic first — the AI has to earn its way in

A generator you can predict

Calendar patterns place 2–6 sessions a week so rest days fall sensibly; focus rotates with your goal; a beginner is never given two high-intensity days in a row.

Progression is data, not hidden code

“Week 2: more reps. Week 3: more volume. Week 4: tighter rest.” Rules are inspectable data, resolved by a pure engine with hard caps — the user can see what week five will ask of them.

Progression fails closed

Autoregulated steps apply only when your logs prove last week’s targets were actually met. No evidence, no increase — and no percentage step may exceed 50%.

The AI lane is optional — and audited

A model may draft the week from the same pre-filtered pool, but the draft is validated structurally and semantically. Any failure falls back to the deterministic engine, invisibly.

2–6	Sets 1–20 · Reps 1–100	≤50%	100%
sessions per week	hard caps in the resolver	max percentage step	of AI drafts validated before use

The plan adapts to the week you actually had

Protect · Maintain · Progress

Four weighted signals from your last 7 days — completion 45%, activity 25%, recency 20%, intake 10% — scale next week's working volume ×0.6, ×1.0 or ×1.1. A hard week quietly eases the next one; a quiet week is never framed as failure.

A guided session logger

Pre-resolved targets, weights rounded to real plates, a rest countdown between sets, actuals recorded set by set — and swaps constrained to genuine alternatives you have equipment for.

Analytics with no AI in it

Weekly volume and set counts per muscle, and a Brzycki estimated one-rep max with the formula version pinned so history can never silently shift.

Achievements are never revoked

Streaks evaluate against your best-ever run, not your current one. A badge earned stays earned — we removed the punishing reset-to-zero counter entirely.

×0.6 / 1.0 / 1.1
protect · maintain · progress

45 / 25 / 20 / 10
readiness signal weights

11
achievement definitions

Progression and readiness, exactly as shipped

apps/api/src/modules/plans/readiness.ts

```ts
/*  completion 45 — the only signal that measures the
 *                  thing we are modulating.
 *  recency    20 — four quiet days in a row read worse
 *                  than four scattered through the week.
 *  intake     10 — smallest weight ON PURPOSE: the
 *                  noisiest signal, and the one most easily
 *                  turned into food moralising. It may nudge
 *                  the band but never decide it.          */
const WEIGHTS = {
  completion: 45, logging: 25, recency: 20, intake: 10,
} as const;

/* Applied to the ABSOLUTE deviation: eating well under
 * target costs exactly as much as eating well over it.
 * Under-eating is never scored as virtue — that is the
 * disordered-eating pattern this product must not reward. */
const INTAKE_ZERO_AT_DEVIATION = 0.3;
```

WHY IT MATTERS Every weight is justified in a comment beside it. A marker can audit our judgement, not just our arithmetic.

apps/api/src/modules/plans/progression.ts

```ts
// Autoregulation: if the logs cannot confirm the previous
// iteration's targets were met, the step is skipped.
if (rule.requires && rule.requires.length > 0) {
  if (!gate || !gate(atIteration, rule)) continue;
}                          // no evidence → no increase

// Hard caps mirror the shared validation schemas:
// sets 1–20, reps 1–100, rest 0–900 s, weight 0–1000 kg.
function clamp(value, kind) {
  return Math.min(CAPS[kind].max,
         Math.max(CAPS[kind].min, value));
}

// Rules are DATA, resolved by this pure engine — so the
// user can inspect week 5, an AI draft can speak the same
// vocabulary, and a unit test can pin the reference example.
```

WHY IT MATTERS The gate fails closed and the caps are unconditional — even a model-drafted plan physically cannot prescribe past them.

Every AI call goes through one door — and survives failure

Lanes, not vendors

Features ask for a capability — “vision”, “chat fast”, “plan structured” — never a brand. The gateway maps each lane onto whichever providers actually have API keys.

Engineered resilience

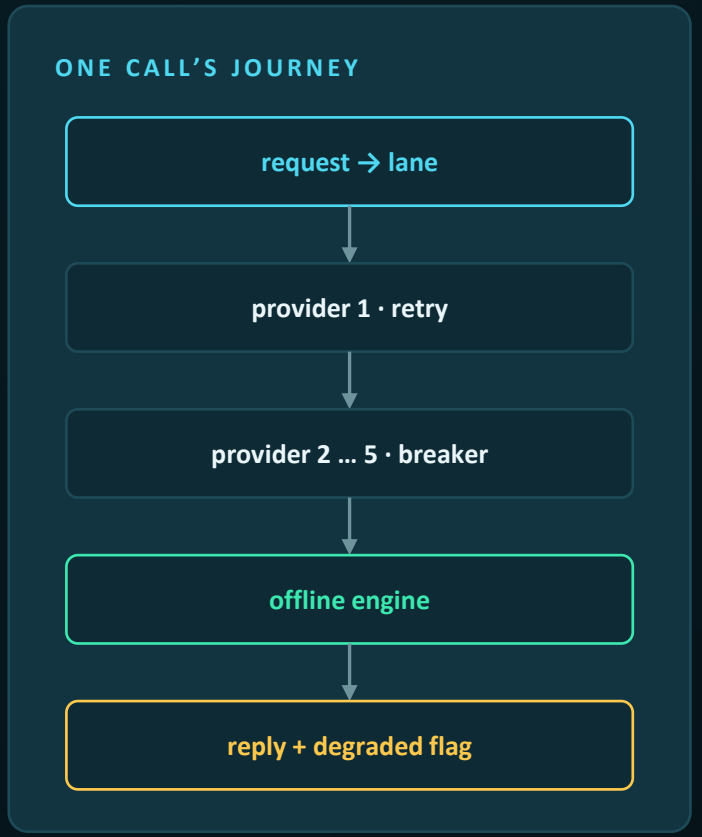
Retries with jittered backoff; a circuit breaker sidelines any provider that fails 3 calls in a row for a 60-second cooldown; one 12-second deadline bounds the whole call.

An offline brain of last resort

If every provider fails — or no keys are configured at all — a deterministic engine answers instead. Every core journey works with the AI completely gone.

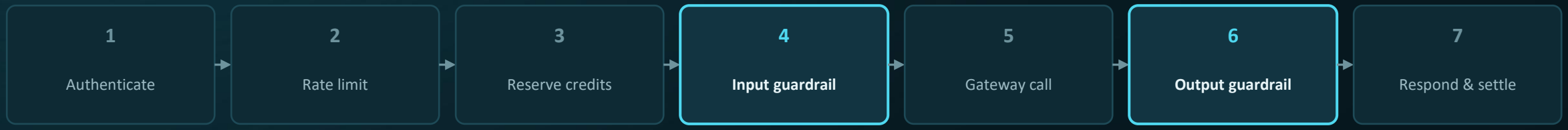
Degraded output is never billed

A fallback answer carries a degraded flag and the reserved credit is returned. Template text is not a model answer anyone should pay for.



5	5	12 s	0
logical model lanes	providers in the chain	overall deadline	credits charged when degraded

Safety is architecture — seven steps in front of every model



The same sequence guards chat, meal photos, meal suggestions, plan generation and the weekly insight. No endpoint is exempt; there are no fast paths.

Crisis always wins

The input classifier prioritises crisis above medical, above extreme diet, above everything. A user in distress never receives diet content — no model is called, and the reply contains real helpline contacts.

Jailbreaks refused, floors enforced

“Pretend you’re not an AI” framing is refused outright. “Help me eat 600 calories a day” is refused on input — and numeric rules re-screen every reply on the way out.

A refused turn is free

Credits are reserved before the guardrail and released when it blocks — so safety never costs the user anything.

5	12	CI-gated
input categories, priority-ordered	versioned prompt files (P-01…P-12)	safety evaluation sets

Locked down by default, private by default

SECURITY

- Access tokens live 15 minutes; refresh tokens are single-use, stored only as hashes
- A replayed refresh token revokes the whole token family — attacker and victim both signed out
- Telegram sign-in: HMAC recomputed server-side, compared in constant time, stale after 10 minutes
- Four rate-limit lanes; five failed logins lock the account for 15 minutes

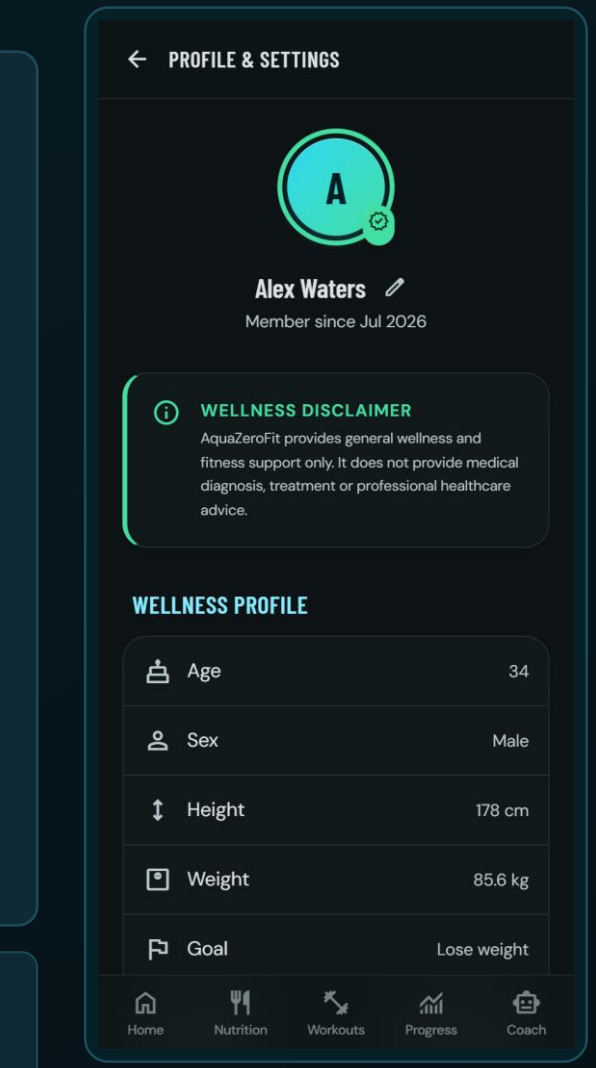
PRIVACY

- All four consents default OFF — with personalisation off, not even your name reaches a model
- AI memory is proposed by the model but approved by you; only approved facts ever enter a prompt
- Full data export in one request
- Two-step deletion: 30-day grace, then a purge that anonymises even audit trails and hashes leftover identifiers

15 min
access-token lifetime

4
consents, all default off

30 days
deletion grace period



Motivation that cannot hurt you, money that cannot drift

XP for behaviour — never outcomes

Logging, training, hydrating and resting earn experience. Nothing scores a calorie deficit or a kilogram lost — that is a safety decision, not a design preference. Capped per lane and at 150 XP a day.

Consistency, not punishing streaks

A 28-day window with a grace day replaces the reset-to-zero streak. The figure cannot count down, and a badge once earned is never revoked.

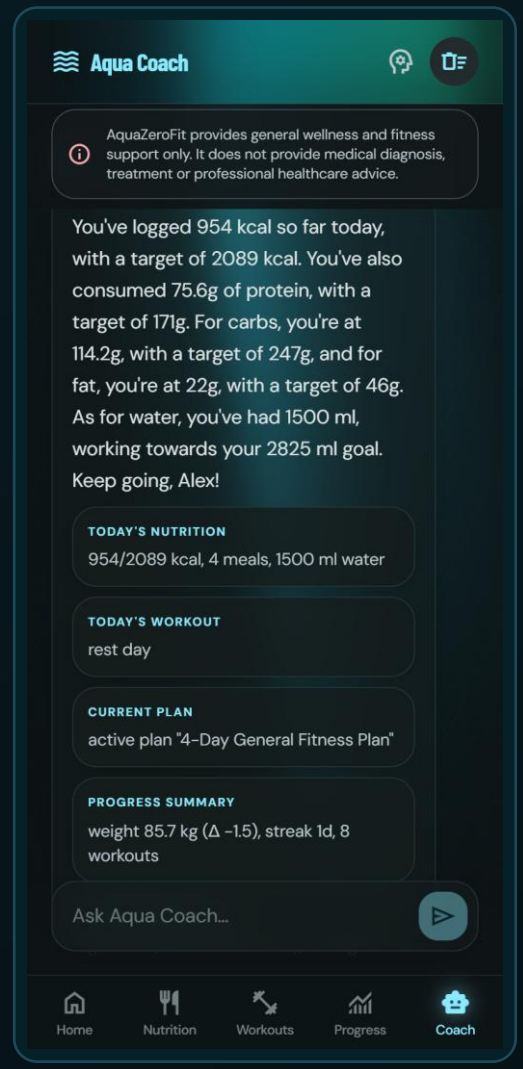
Nine coach personas, honestly gated

Tournament fighters as coaching voices, unlocked by level. Telegram Stars is only a shortcut — and the server, never the client, decides every entitlement and price.

An append-only credit ledger

AI usage is metered by transactions — reserve, commit, release — and the balance is computed from history. A counter can drift and cannot be audited; a ledger can.

150	28 + 1	9	50
max XP per day	consistency window + grace day	coach personas	free AI credits daily



The gateway’s last line, and the classifier’s first

apps/api/src/modules/ai/gateway.ts

```
outer: for (const provider of PROVIDERS) {
  const credentialed = !!process.env[provider.keyEnv];
  if (circuitIsOpen(provider.name)) continue;
  ...                       // retries w/ jittered backoff,
                            // one 12 s deadline overall
}

// Deterministic offline engine — the product must work
// with zero keys. It is never retried and never timed out:
// the eval runner and test suite depend on its output
// being byte-identical.
const degraded = realProviderInPlay;
const degradedReason = degraded
  ? deadlineExceeded ? 'deadline_exceeded'
                     : 'provider_failure'
  : undefined;    // degraded ⇒ credit released
```

WHY IT MATTERS Application code never names a vendor. Unplug every API key and this loop falls through to an engine that always answers — free of charge.

apps/api/src/modules/ai/guardrails.ts

```
/** Pure classifier — no I/O, usable from tests and eval
  * runners. Priority: crisis > medical > extremeDiet. */
const crisis = findMatch(text, CRISIS_PATTERNS);
if (crisis) return { category: 'crisis', ... };

const medical = findMatch(text, MEDICAL_PATTERNS);
if (medical) return { category: 'medical', ... };

if (hasSubFloorTarget(text)) {   // "600 kcal a day"
  return { category: 'extremeDiet', ... };
}

// A pure instruction-override attempt with no unsafe
// payload is still refused: the assistant never
// renegotiates its scope.
if (jailbreak) return { category: 'outOfScope', ... };
```

WHY IT MATTERS The priority order IS the code order — crisis returns first, so nothing can pre-empt it. This function runs before any credit is spent on a model.

How we know all of this is true

770

automated tests, all passing

614 + 156

API tests + web tests

12

versioned prompt files

122

code listings in our report, verified byte-exact

Tests that defend decisions, not just lines

A test fails if anyone merges the coach persona into the rules prompt, if a refresh token stops revoking its family, or if a safety evaluation set regresses. The pipeline is gated on all of it.

A service that refuses to boot broken

In production it will not start without real secrets, HTTPS-only origins and a working mail transport — misconfiguration becomes a loud failure, never a silent one.

Honest about limits

Single-instance scaling and the pure-JS password hash are recorded as deliberate trade-offs with a stated hardening path — decided early, not discovered late.

```
Run it yourself:  npm run verify
```

LIVE DEMONSTRATION | PRESENTER: WHOLE TEAM

Three minutes, seven screens — each one proves something

1	**Dashboard with the calorie ring**	Targets are personalised and today’s arithmetic is live
2	**Photograph a meal — and pause on the draft**	Nothing is saved yet; the calories on screen were computed by the app, not the model
3	**Adjust a portion, then confirm**	The user controls what enters their record
4	**Today’s workout: one set + the rest timer**	The plan resolves per day and records what you actually did
5	**Ask the coach about today**	Grounding — the answer contains the user’s real numbers
6	**Ask the coach something medical**	The safety boundary, live: a warm refusal and a signpost to real help
7	**The progress screen**	What changed and why — a sentence, not just a chart

If the network fails, nothing changes: the offline engine answers and every journey still completes.

Questions — ask the person who built it

VH — **VICTOR HONG**

Nutrition & Dietary System

- Calorie & macro engine, the 1,200 kcal floor
- Photo pipeline & the confirmation gate
- Barcodes, allergens, recipes, export

EL — **ERIC LA**

Training & Workout System

- Exercise corpus & licence governance
- Plan generator & progression-as-data
- Readiness, session logger, analytics

TW — **TUNDJI WILLIAMS-FULWOOD**

Platform, AI, Safety & Operations

- AI Gateway, guardrails, offline fallback
- Auth, privacy, consent & deletion
- Gamification, payments, testing, ops

BUILT ON THE SHOULDERS OF

Data: wger exercise database (CC BY-SA 4.0, attribution rendered in-app) · Open Food Facts (ODbL) · USDA FoodData Central (CC0)

Platform: Telegram Bot & Mini App platform · React 18 · Node.js + Express · TypeScript · zod · Tailwind CSS · TanStack Query · Vitest · sharp

Project: AquaZeroFit v1.0.0 · licensed AGPL-3.0-or-later · full licence and attribution register in the repository (ATTRIBUTION.md, THIRD_PARTY_NOTICES.md)

Thank you.

APPENDIX

Code deep dives for Q&A

Kept out of the ten minutes on purpose. Every listing on the next slides is real shipped code — quoted from the repository and documented, byte-exact, in sections 9–11 of our Software Architecture and Contribution Report (AQF-24).

- **Meal-photo privacy: the re-encode** Victor Hong
- **Stolen sessions die on replay** Tundji Williams-Fulwood
- **Balances that cannot drift** Tundji Williams-Fulwood

One line that is both the privacy measure and the type check

apps/api/src/modules/vision/router.ts

```ts
/**
 * Decode the upload and write a fresh JPEG from the pixels.
 *
 * Privacy: a phone photo carries EXIF GPS at home-address precision, the capture timestamp and the
 * camera serial number. Persisting that verbatim would bolt a location trail onto health-adjacent
 * records that only need the picture of the food. Re-encoding rebuilds the file from decoded pixels,
 * so every metadata block (EXIF, XMP, IPTC, ICC) is discarded.
 *
 * Security: this is also the real upload type check — `file.mimetype` is an unverified client claim.
 */
async function toStorableJpeg(buffer: Buffer, declaredMime?: string): Promise<Buffer> {
  return await sharp(buffer).rotate().jpeg({ quality: 82 }).toBuffer();
}
```

WHY IT MATTERS If the bytes are not really an image, sharp throws and the upload is rejected. The photo itself is deleted on confirm — or swept within 24 hours.

Likely Q&A pairing: “How do you protect a user’s photos?” — and the answer is a file path, not a promise.

A stolen session dies the moment it is replayed

apps/api/src/platform/auth.ts

```
/**
 * Rotate a refresh token: single use. A second presentation of the same token is treated as
 * theft — the entire family is revoked (AQF-07 §2.1). Atomic compare-and-swap ensures only one
 * concurrent refresh can succeed.
 */
if (existing.usedAt !== null || existing.revokedAt !== null) {
  revokeFamily(existing.familyId);
  throw new AppError('AUTH_INVALID', 'Refresh token reuse detected; session revoked');
}

// Atomic CAS: mark token as used only if still unused.
const marked = await store.compareAndSwapRefreshToken(existing.id, tokenHash, new Date().toISOString());
if (!marked) { revokeFamily(existing.familyId); throw new AppError('AUTH_INVALID', ...); }
```

WHY IT MATTERS Attacker and victim hold the same family — revoking it signs BOTH out, which is exactly the property you want when you cannot tell which one is the thief.

Likely Q&A pairing: “What happens if someone steals a token?” — tokens are hashed at rest, live 15 minutes, and reuse kills the family.

Balances that cannot drift, and an SDK that earns its load

apps/api/src/modules/ai/creditLedger.ts

```ts
/**
 * Append-only credit ledger. Every movement is a
 * CreditTransaction document; balance is a plain fold
 * (sum of amounts) over the user's transactions:
 *   grant / purchase  → +amount
 *   reserve           → -cost   (hold deducted now)
 *   release           → +cost   (cancels a hold)
 */

/** Balance = fold. No cached counters, ever. */
async balance(userId: string): Promise<number> {
  const txs = await userTxs(userId);
  return txs.reduce((sum, t) =>
    sum + (typeof t.amount === 'number' ? t.amount : 0), 0);
}
```

WHY IT MATTERS A crashed job returns its hold; an auditor can replay history to any balance. Experience points obey the same contract.

apps/web/src/lib/telegram.ts

```ts
/** Does this page load look like a Telegram Mini App
  * launch? Answered WITHOUT the SDK, so it can decide
  * whether to fetch it at all. */
export function looksLikeTelegramLaunch(): boolean {
  if (getWebApp()?.initData) return true;
  const surface = `${location.hash}${location.search}`;
  if (/[#&?]tgWebApp(Data|Version|Platform)=/.test(surface))
    return true;
  ...
}

// The SDK used to be a blocking <script> fetched from
// telegram.org on every page load — stalling the marketing
// site for exactly the corporate networks that block
// telegram.org, whose users need the browser fallback most.
```

WHY IT MATTERS The comment records the cost that drove the design. That is what a decision register looks like inside the code.